

SISTEMAS PROGRAMABLES

Reporte de práctica

Integrantes del equipo:

Daniel Bustamante

Iván Luan

Materia: Sistemas Programables

Plataforma: Arduino UNO R4 WiFi

1. Introducción

En un sistema de conteo de piezas sobre una banda transportadora, el sensor de entrada debe detectar cada pieza sin interferir con otras tareas del microcontrolador, como la animación de una matriz LED. Este reporte documenta la conexión y el funcionamiento de un pulsador utilizado como sensor, leído mediante una interrupción externa en un Arduino UNO R4 WiFi, así como la lógica de software que garantiza una detección confiable de cada evento.

2. Objetivo

Conectar un pulsador como sensor de piezas usando el mínimo número de cables posible, aprovechando la resistencia pull-up interna del microcontrolador, y documentar el uso de una interrupción externa (FALLING) junto con un buffer circular por software para registrar los eventos sin bloquear la animación de la matriz LED.

3. Materiales

- 1 Arduino UNO R4 WiFi
- 1 pulsador (push-button) de 4 patas
- 2 cables de conexión (jumpers)
- 1 matriz LED (controlada por el mismo sketch, animación no bloqueante)

4. Diagrama de conexión

La conexión utilizada es mínima: solo dos cables, sin resistencia externa ni fuente de alimentación adicional.

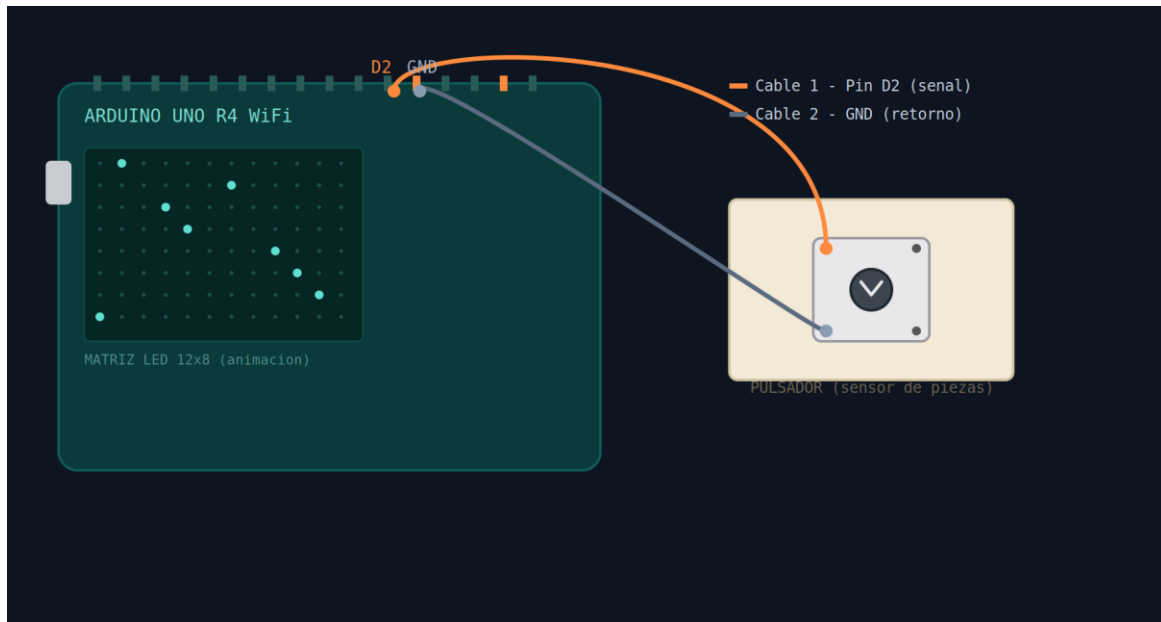


Figura 1. Diagrama de cableado del pulsador conectado a D2 y GND.

Solo dos conexiones: D2 → pata A del pulsador · GND → pata B (diagonal opuesta) del pulsador.

5. Tabla de conexiones

Cable	Arduino UNO R4 WiFi	Pulsador	Función
Cable 1 (naranja)	D2	Pata A	Señal digital leída por la interrupción externa (FALLING)
Cable 2 (gris)	GND	Pata B (diagonal opuesta a A)	Retorno a tierra; cierra el circuito al presionar

Nota: en un pulsador de 4 patas, las patas quedan unidas en pares por dentro (A con la de su mismo lado, B con la del lado contrario). Por eso se debe usar una pata de cada lado del botón (diagonalmente opuestas), de manera que el circuito solo se cierre al presionar y no de forma permanente.

6. Por qué funciona con solo dos cables

El pin D2 se configura como INPUT_PULLUP, lo que activa internamente una resistencia de pull-up que conecta el pin a 5 V dentro del propio microcontrolador. En reposo, sin ninguna conexión externa adicional, D2 se lee en estado HIGH.

Al presionar el pulsador, D2 queda unido directamente a GND y su lectura cae a LOW. Ese flanco de bajada (HIGH → LOW) es precisamente la condición que dispara `attachInterrupt(..., FALLING)`. Gracias a esto, no

se necesita una resistencia externa ni una fuente de alimentación adicional: la pull-up interna del R4 y el GND de la propia placa son suficientes para que el circuito funcione de forma confiable.

7. Qué hace el sketch con esa señal

El firmware separa claramente la captura del evento (rápida, dentro de la interrupción) del procesamiento del evento (dentro del loop principal), evitando bloquear en todo momento la animación de la matriz LED:

- ISR `piezaDetectada()`: filtra rebotes mecánicos (50 ms) y guarda el instante `millis()` en un buffer circular de 16 posiciones.
- Buffer circular: usa dos contadores que solo crecen (escritos / leídos) en lugar de un puntero que da la vuelta, evitando ambigüedad entre "lleno" y "vacío".
- `loop()`: nunca se detiene animando la matriz LED; solo entra a `noInterrupts()` por un instante brevísimo para copiar el contador de forma segura.
- Detección de overflow: si se acumulan más de 16 eventos sin procesar, se detecta la condición y se avisa por el monitor serie.

7.1 Captura del evento (interrupción)

```
const uint8_t PIN_PULSADOR = 2;
const unsigned long T_DEBOUNCE = 50;    // ms
const uint8_t TAM_BUFFER = 16;

volatile unsigned long bufferTiempos[TAM_BUFFER];
volatile uint16_t contadorEscritos = 0;  // solo crece
volatile uint16_t contadorLeidos = 0;    // solo crece
volatile unsigned long ultimaInterrupcion = 0;

void piezaDetectada() {
    unsigned long ahora = millis();

    // Filtro de rebote: ignora flancos demasiado cercanos entre si
    if (ahora - ultimaInterrupcion < T_DEBOUNCE) return;
    ultimaInterrupcion = ahora;

    // Buffer circular por indice modulo, con contadores que solo crecen
    bufferTiempos[contadorEscritos % TAM_BUFFER] = ahora;
    contadorEscritos++;
}

void setup() {
    Serial.begin(115200);
    pinMode(PIN_PULSADOR, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(PIN_PULSADOR), piezaDetectada, FALLING);
}
```

7.2 Procesamiento no bloqueante en loop()

```
void loop() {
    // La animacion de la matriz LED nunca se detiene: se actualiza
    // por millis(), igual que el semaforo de practicas anteriores.
```

```

actualizarAnimacionMatriz();

// Copia segura del contador de escritura (seccion critica breve)
uint16_t escritosLocal;
noInterrupts();
escritosLocal = contadorEscritos;
interrupts();

// Deteccion de overflow: mas de TAM_BUFFER eventos sin procesar
if ((uint16_t)(escritosLocal - contadorLeidos) > TAM_BUFFER) {
    Serial.println("ADVERTENCIA: overflow del buffer de piezas");
    contadorLeidos = escritosLocal - TAM_BUFFER; // resincroniza
}

// Procesa todas las piezas pendientes en el buffer
while (contadorLeidos < escritosLocal) {
    unsigned long t = bufferTiempos[contadorLeidos % TAM_BUFFER];
    contadorLeidos++;
    Serial.print("Pieza #");
    Serial.print(contadorLeidos);
    Serial.print(" detectada en t = ");
    Serial.println(t);
}
}

```

8. Pasos de montaje

1. Ubica el pulsador de forma que sus 4 patas queden a caballo sobre el canal central de la protoboard (si se usa una), separando los dos pares internos.
2. Cable 1 (naranja): del pin D2 del Arduino a una pata del pulsador.
3. Cable 2 (gris): de GND del Arduino a la pata diagonalmente opuesta del pulsador.
4. Carga el sketch. En reposo se verá la animación serpenteante en la matriz; cada pulsación imprime "Pieza #N detectada..." en el monitor serie sin que la animación se detenga.

9. Conclusiones

El uso de INPUT_PULLUP permitió reducir el cableado del sensor a solo dos conexiones, eliminando la necesidad de resistencias externas. Combinar una interrupción externa con un buffer circular de dos contadores garantizó que ninguna pieza detectada se perdiera, incluso si el conteo ocurre mientras el loop() está ocupado animando la matriz LED. Esta separación entre captura (ISR) y procesamiento (loop) es una práctica clave en sistemas programables donde deben convivir eventos rápidos e impredecibles (como el paso de una pieza) con tareas continuas de refresco visual.

Como equipo, esta práctica reforzó la importancia de diseñar el manejo de interrupciones de forma mínima y segura (ISR cortas, secciones críticas breves), evitando bloqueos y pérdidas de datos en aplicaciones de conteo en tiempo real.